Legare statică/dinamică – static/dynamic binding în Java

Programare Orientată pe Obiecte

Polimorfism

- **Polimorfism – abilitatea unui obiect de a se comporta diferit la acelaşi mesaj**
- **Două tipuri: static şi dinamic**

- **Supraîncărcarea (overloading)**
- **Supradefinirea (overriding)**

```
class A {
    void metoda() {
        System.out.println("A: metoda fara parametru");
    }
    // Supraîncărcare
    void metoda(int arg) {
        System.out.println("A: metoda cu un parametru");
    }
}
class B extends A {
    // Supradefinire
    void metoda() {
        System.out.println("B: metoda fara parametru");
    }
}
```

Static/dynamic binding în Java

- Binding – procesul de a stabili ce metodă sau variabilă va fi apelată

- **Static binding şi dynamic binding - două concepte importante în Java**
- Legate direct de execuţia codului
- Mai multe metode cu acelaşi nume (method overriding) sau două variabile cu acelaşi nume în aceeaşi ierarhie de clase, care este utilizată?
- Majoritatea referinţelor sunt rezolvate în timpul compilării, dar cele care depind de obiect şi de polimorfism sunt rezolvate la execuţie, atunci când este de fapt disponibil obiectul.

- Dynamic binding – late binding (la execuţie)
- Static binding – early binding (la compilare)

Static/dynamic binding în Java

- *Diferenţa între static şi dynamic binding* în Java
- Întrebare populară la angajarea în domeniu
- Explorează cunoştinţele candidaţilor legate de cine determină ce metodă va fi apelată dacă există mai multe metode cu acelaşi nume, ca în cazul metodelor supraîncărcate sau supradefinite (method overloading and overriding).
- Compilatorul sau JVM – maşina virtuală Java?

Legare statică/dinamică – Static/dynamic binding în Java

```
class Vehicle {
    public void drive() {
        System.out.println("A");
    }
}

class Car extends Vehicle {
    public void drive() {
        System.out.println("B");
    }
}
class TestCar {
    public static void main(String args[]) {
        Vehicle v;
        Car c;
        v = new Vehicle();
        c = new Car();
        v.drive();
        c.drive();
        v = c;
        v.drive(); // ?
    }
}
```

Output

A

B

B

Polimorfism

- Legarea dinamică are loc în Java pentru orice metodă care nu are atributul *final* sau *static*, metodă numită polimorfică.
- In Java majoritatea metodelor sunt polimorfice.
- Metodele polimorfice Java corespund functiilor virtuale din C++.
- Apelul functiilor polimorfice este mai putin eficient ca apelul functiilor cu o singura formă.
- Fiecare clasă are asociat un tabel de pointeri la metodele (virtuale) ale clasei.

Static Binding vs Dynamic binding în Java

Diferenţe între legarea statică şi cea dinamică în Java:

- **Static binding** se realizează la **Compilare** în timp ce **Dynamic binding** se realizează la **Rulare** - Execuţie.

- Metodele şi variabilele **private**, **final** sau **static** utilizează static binding în timp ce metodele **virtuale - abstracte** utilizează dynamic binding.

- **Static binding** se face pe baza informaţiei legate de **Tip** (**clasa** în Java), în timp ce **Dynamic binding** utilizează **Obiectul** pentru a realiza legarea.

- **Static binding** este folosit frecvent la metodele supraîncărcate (**overloaded** methods), **Dynamic binding** (dynamic dispatch) este asociat în general cu metodele supradefinite (**overriding** methods).

Static Binding: Exemplu Java

```
public class StaticBindingTest {
    public static void main(String args[])  {
            StaticBindingTest et = new StaticBindingTest();
            Collection c = new HashSet();
            et.sort(c);
     }
  // metoda supraincarcata cu argument Collection
    public Collection sort(Collection c){
            System.out.println("In metoda sort(Collection)!");
            return c;
       }
   /*metoda supraincarcata cu argument HashSet, subtip al
    lui Collection */
    public Collection sort(HashSet hs){
            System.out.println(“In metoda sort(HashSet )!");
            return hs;
       }
    }
```

Output: In metoda sort(Collection)!

Dynamic Binding: Exemplu Java

```
class Vehicle {
    public void start() {
            System.out.println("In metoda start din Vehicle!");
    }
}
class Car extends Vehicle {
    public void start() {
            System.out.println ("In metoda start din Car!");
    }
}
public class DynamicBindingTest {
  public static void main(String args[]) {
     Vehicle vehicle = new Car(); //tip Vehicle, dar obiect Car
     vehicle.start();   //start din clasa Car - start() e supradef
    }
}
```

Output: In metoda start din Car!

Dynamic Binding

- Conceptul de overriding
- Car extends Vehicle - supradefineşte start()
- Apelul lui start() pentru un obiect Vehicle, apelează start() din subclasa Car deoarece obiectul referit de tipul Vehicle este un obiect Car
- Aceasta se petrece la execuţie pentru că obiectul este creat doar la execuţie -> Dynamic binding in Java.
- Dynamic binding este mai încet decât static binding pentru că apare în momentul execuţiei şi necesită timp pentru a determina care metodă este apelată de fapt.

Exemplu Static Binding vs Dynamic binding

```
public class Animal {
        public String type = "mamifer";
        public void show() {
            System.out.println("Animalul este un: " + type);
        }
}
public class Dog extends Animal {
        public String type;
        public Dog(String type){
            this.type = type;
        }
        public void show() {
            System.out.println("Cainele este un: " + type);
        }
}
```

```
…
Animal doggie = new Dog("ciobanesc");
 doggie.show();
System.out.println ("Tipul este: " + doggie.type);
....
```

Exemplu Static Binding vs Dynamic binding

Output:

Cainele este un: ciobanesc
(dynamic binding)
Tipul este: mamifer
(static binding)

Ce se afiseaza daca nu am avea in clasa Dog:

public String type;

Output:

Cainele este un: ciobanesc
Tipul este: ciobanesc

Observaţii

- Datele nu sunt niciodată supradefinite, doggie.type foloseşte Animal.type -- "static binding“

Avantaj: comportamentul este legat de tipul obiectului invocat, fără ca noi să ştim precis tipul acestuia.

Exemplu: Daca trimitem un Animal, nu ştim dacă este Cat/Dog/altceva, dar el va adopta comportamentul adecvat:

```
public void makeNoise(List<Animal> animals) {
      for (Animal a : animals) {
         a.makeNoise();
      }
}
```

- Fiecare animal din lista va produce propriul zgomot (va mieuna, lătra, etc)
- **Observaţie**: Clasa Animal poate fi abstracta. Astfel, comportamentul va fi definit de clasele concrete derivate din ea.

Exemplu

```
public class Shape {
    int x=10;
    void draw() {
        System.out.println(“Shape”);
    }
}
public class Circle extends Shape{
    int x=5;
    void draw(){
        System.out.println(“Circle”);
    }
}
public class Test{
    public static void main(String[] args) {
        Shape shape = new Circle();
        shape.draw();  // DYNAMIC BINDING
        System.out.println(shape.x);  // STATIC BINDING
    }
}
```

Explicaţii

- Atunci când compilatorul vede apelul draw(), el ştie că obiectul respectiv este de tipul Shape, dar el ştie şi că acel obiect poate fi o referinţă la orice clasă derivată din Shape.
- De aceea, compilatorul nu ştie ce versiune a metodei draw() este de fapt apelată. Aceasta se va şti doar în momentul execuţiei instrucţiunii respective:
- **shape.draw();** - metoda draw din Circle

- În unele cazuri, compilatorul poate determina versiunea corectă
- În Java, variabilele membru prezintă static binding, deoarece Java nu permite comportament polimorfic pentru variabilele membru.
- Aceasta înseamnă că atât clasa Shape cât şi clasa Circle au o câte o variabilă membru cu acelaşi nume:

System.out.println(shape.x); - valoarea lui x din Shape.

Concluzie - dynamic binding

- metodele suprascrise în clasele derivate vor fi apelate folosind dynamic binding (late binding).
- este un mecanism prin care compilatorul, în momentul în care nu poate determina implementarea unei metode în avans, lasă la latitudinea JVM-ului (maşinii virtuale) alegerea implementării potrivite, în funcţie de tipul real al obiectului.
- această legare a implementării de numele metodei la **momentul execuţiei** stă la baza polimorfismului.

Exerciţiu propus

```
class Ana {
        public void print( Ana p) {
            System.out.println("Ana 1\n");
        }
}
class Mihai extends Ana {
        public void print (Ana p) {
            System.out.println("Mihai 1\n");
        }
        public void print (Mihai l) {
            System.out.println("Mihai 2\n");
        }
}
class Dana extends Mihai {
        public void print (Ana p) {
            System.out.println("Dana 1\n");
        }
        public void print (Mihai l) {
            System.out.println("Dana 2\n");
        }
        public void print (Dana b) {
            System.out.println("Dana 3\n");
        }
}
```

Exerciţiu propus

```
public class Test{
    public static void main (String [] args) {
        Mihai stud1 = new Dana();
        Ana stud2 = new Mihai();
        Ana stud3 = new Dana();
        Dana stud4 = new Dana();
        Mihai stud5 = new Mihai();
 1      stud1.print(new Ana());
 2      ((Dana)stud1).print(new Mihai());
 3      ((Mihai)stud2).print(new Ana());
 4      stud2.print(new Dana());
 5      stud2.print(new Mihai());
 6      stud3.print(new Dana());
 7      stud3.print(new Ana());
 8      stud3.print(new Mihai());
 9      ((Dana)stud3).print(new Mihai());
10      ((Dana)stud3).print(new Dana());
11      stud4.print(new Dana());
12      stud4.print(new Ana());
13      stud4.print(new Mihai());
14      stud5.print(new Dana());
15      stud5.print(new Mihai());
16      stud5.print(new Ana());   } }
```

Ierarhie

Ana – print (Ana)

|

Mihai – print (Ana), print (Mihai)

|

Dana – print (Ana), print (Mihai), print (Dana)

Tip – nume -> obiect

- **Mihai - stud1 -> Dana**
- **Ana - stud2 -> Mihai**
- **Ana - stud3 -> Dana**
- **Dana - stud4 -> Dana**
- **Mihai - stud5 -> Mihai**

Genericitate

Programare Orientată pe Obiecte

Introducere

- concept nou - JDK 5.0.
- oferă un mijloc de **abstractizare a tipurilor de date**
- util mai ales în ierarhia de colecţii
- din unele puncte de vedere, se poate asemăna cu conceptul de *template* din C++.
- permite parametrizarea tipurilor de date (clase și interfețe), parametrii fiind tot tipuri de date
- Control sporit asupra lucrului cu tipuri de date, verificarea tipului de date se face la compilare
- Eliminarea operatorului de cast
- Scrierea de algoritmi generici

Fără genericitate

Exemplu:

```
List myList = new ArrayList();
myList.add(new Integer(0));
Integer x = (Integer) myList.get(0);
```

Obs: necesitatea operaţiei de cast pentru a identifica corect variabila obţinută din listă.

Dezavantaje:

- Este îngreunată citirea codului
- Apare posibilitatea unor erori la execuţie
  - în momentul în care în listă se introduce un obiect care nu este de tipul *Integer*.

- *Genericitatea* intervine pentru a elimina aceste probleme!

Tipuri generice (parametrizate)

- **Tipizarea elementelor unei colecţii:**
  TipColectie < TipDeDate >

```
// Inainte de 1.5
    List list = new ArrayList();
    list.add(new Integer(123));
    int val = ((Integer)list.get(0)).intValue();

// Dupa 1.5, folosind si autoboxing
    List<Integer> list = new ArrayList<Integer>();
    list.add(123);
    int val = list.get(0);
```

- **Avantaje**: simplitate, control (eroare la compilare vs. ClassCastException)

Tipuri generice (parametrizate)

Observatii:

- lista nu mai conţine obiecte oarecare, ci poate conţine doar obiecte de tipul *Integer*.
- a dispărut şi cast-ul.
- **verificarea tipurilor este efectuată de compilator**, ceea ce elimină potenţialele erori de execuţie cauzate de eventuale cast-uri incorecte.

Beneficiile utilizării genericităţii:

- îmbunătăţirea lizibilităţii codului
- creşterea gradului de robusteţe

Observaţii

- Deoarece colecţiile sunt construite peste tipul de date Object, metodele de tip *next* sau *prev* ale iteratorilor vor returna tipul Object, fiind responsabilitatea noastră de a face conversie la alte tipuri de date, dacă e cazul!

```
ArrayList<Integer> list=new ArrayList<Integer>();
for (Iterator i = list.iterator(); i.hasNext();) {
    Integer val=(Integer)i.next();
    ...
}
```

sau:

```
ArrayList<Integer> list=new ArrayList<Integer>();
for (Iterator <Integer> i = list.iterator(); i.hasNext();) {
    Integer val=i.next();
    ...
}
```

sau:

```
ArrayList<Integer> list=new ArrayList<Integer>();
for (Integer val : list) {
    ...
}
```

Exemple

```
Collection<String> c = new ArrayList<String>();
c.add("Test");
c.add(2); // EROARE la complare!
Iterator<String> it = c.iterator();
while (it.hasNext()) {
   String s = it.next();
}
```

- O **iterare** obişnuită pe un map de perechi (String, Student) se va face în felul următor:

```
for (Map.Entry<String,Student> entry:
   students.entrySet() )
   System.out.println("Media “+ entry.getKey()+“este“
   +entry.getValue().getAverage());
```

- bucla for-each⇔ iteratorul mulţimii de perechi întoarse de entrySet.

Definirea unui tip generic

class ClassName<T1, T2, ..., Tn> { … }

sau

interface IName<T1, T2, ..., Tn> { … }

```
/**
* Versiunea generica a clasei Stack
* @param <E> tipul elementelor
*/
public class Stack<E> {
    // E reprezinta un tip generic de date
    private E[ ] items;
    public void push(E item) { ... }
    public E peek() { .. }
}
```

Convenții de numire a parametrilor de tip

- E - Element (folosit intensiv de Java Collections Framework)
- K - Key
- N - Number
- T - Type
- V - Value
- S,U,V etc. - 2nd, 3rd, 4th types

Exemple:

public class **Node<T>** { … }

public interface **Pair<K, V>** { … }

public class PairImpl<K, V> implements Pair<K, V> {...}

Definirea unor structuri generice simple

- Exemple - definiţia oferită de Java pentru tipurile *List* şi *Iterator*.

```
public interface List<E> {
    void add(E x);
    Iterator<E> iterator();
}
public interface Iterator<E> {
    E next();
    boolean hasNext();
}
```

- Sintaxa <E> - folosită pentru a defini ***tipuri formale*** în cadrul interfeţelor.
- În momentul în care invocăm efectiv o structură generică, ele vor fi înlocuite cu tipurile efective utilizate în învocare.

Instanțierea tipurilor generice

- Invocarea generică a unui tip

```
Stack<String> stack = new Stack<String>();
Pair<Integer,String> pair =
new PairImpl<Integer,String>(0, "ab");
Stack<Node<Integer>> nodes = new
    Stack<Node<Integer>>();
```

- Operatorul *diamond* <>

De la versiunea 7, atata timp cat compilatorul poate determina tipul argumentelor din context

```
Stack<String> stack = new Stack<>();
Pair<Integer,String> pair = new PairImpl<>(0, "ab");
Stack<Node<Integer>> nodes = new Stack<>();
```

Instantierea unor structuri generice simple

Exemplu:

```
ArrayList<Integer> myList = new
    ArrayList<Integer>();
Iterator<Integer> it = myList.iterator();
```

- În această situaţie, tipul formal *E* a fost înlocuit (la compilare) cu tipul efectiv *Integer*.

- Observaţie: analogie cu parametrii funcţiilor: se definesc utilizând parametri *formali*, urmând ca, în momentul unui apel, aceşti parametri să fie înlocuiţi cu parametrii *actuali*.

Raw Types

= Numele unei clase sau interfete generice fara argumente de tip

Exemplu:

```
public class Box<T> {
        public void set(T t) { /* ... */ }
        // ...
}
```

- Pt a crea un tip parametrizat pentru Box<T>:

  ```
  Box<Integer> intBox = new Box<>();
  ```

- Daca argumentul de tip este omis => un tip de baza (raw type) pt Box<T>:

  ```
  Box rawBox = new Box();
  ```

- Asadar, Box este tipul de baza (raw type) pentru tipul generic Box<T>
- Atentie! Un tip corespunzator unei clase non-generice sau unei interfete nu este un raw type!
- Raw types – apar pentru ca multe din clasele API nu au fost generice pana la versiunea JDK 5.0.
- Cand utilizam raw types, obtinem comportamentul pre-generic!

Raw types

- Pentru compatibilitate, este permisa atribuirea unui tip parametrizat tipului sau de baza:

```
Box<String> stringBox = new Box<>();
Box rawBox = stringBox; // OK
```

- Dar, invers apare warning:

```
Box rawBox = new Box();
      // rawBox is a raw type of Box<T>
Box<Integer> intBox = rawBox;
      // warning: unchecked conversion
```

- Apare warning si daca folosim un raw type pentru a invoca metode generice

```
Box<String> stringBox = new Box<>();
Box rawBox = stringBox;
rawBox.set(8);
      // warning: unchecked invocation to set(T)
```

- Indicatie: A se evita utilizarea raw types.

Genericitatea în subtipuri

Exemplu:

```
List<String> stringList = new ArrayList<String>();
                                        // corect!
List<Object> objectList = stringList;
                                        // operaţie corectă?
```

Presupunem că operaţia e corectă =>

- am putea introduce în *objectList* orice fel de obiect, nu doar obiecte de tip String =>
- Potenţiale erori de execuţie.
- Exemplu

```
objectList.add(new Object());
String s = objectList.get(0); // operatie ilegala!
```

=> Operaţia **nu va fi permisă de către compilator**!

Observaţie importantă

- Integer extends Object
- Integer extends Number

Dar nu:

- **Stack<Integer> extends Stack<Object>**
- **Stack<Integer> extends Stack<Number>**

Dacă *SubType* este un subtip

- (clasă descendentă sau
- subinterfaţă)

al lui *SuperType:*

Atenţie!

O structură generică:

GenericStructure <SubType>

NU este un subtip al lui:

GenericStructure < SuperType > !

Tip – Subtip in genericitate - concluzie

- Fie două tipuri de date A și B (de exemplu, *Number* și *Integer*)

- Indiferent dacă A si B sunt în relație de moștenire sau nu, *MyClass<A>* nu este în nici o relație cu *MyClass<B>!*

- Părintele comun al claselor *MyClass<A>* și *MyClass<B>* este *Object*!

“This is a common misunderstanding when it comes to programming with generics,

but it is an important concept to learn”